



Article

DLR-Auth: A Decentralized Lightweight and Revocable Authentication Framework for the Industrial Internet of Things

Yijia Dai, Yitong Li , Ye Yuan, Xianwei Gao *, Cong Bian and Meici Liu

Beijing Electronic Science and Technology Institute, Beijing 100070, China; 20242906@mail.besti.edu.cn (Y.D.); 20243915@mail.besti.edu.cn (Y.L.); yuanye@besti.edu.cn (Y.Y.); 20242922@mail.besti.edu.cn (C.B.); 20242940@mail.besti.edu.cn (M.L.)

* Correspondence: gaoxianwei@besti.edu.cn

Abstract

The integration of operational technology (OT) and information technology (IT) within the Industrial Internet of Things (IIoT) has posed prominent security challenges for resource-constrained devices. Existing authentication architectures often suffer from critical vulnerabilities: one is their reliance on centralized trusted third parties, which creates single points of failure; the other is their use of static credentials like biometrics, which pose severe privacy risks if compromised. To address these limitations, this paper proposes DLR-Auth, which combines chaotic synchronization of semiconductor superlattice physically unclonable functions (SSL-PUFs) with Shamir's secret sharing (SSS) to enable decentralized registration and revocable templates. Notably, DLR-Auth is a two-party authentication framework that removes the need for a separate online registration authority that operates directly between a user device (UD_i) and a server (S). In our setting, the server S still acts as the central relying party and hardware authority embedding the matched SSL-PUF module. The protocol also includes an efficient multi-access mechanism optimized for high-frequency interactions. Formal security analysis with the Real-or-Random (ROR) model proves the semantic security of the session key, while performance evaluations demonstrate that DLR-Auth has significant advantages in computational and communication efficiency. DLR-Auth thus offers a robust, scalable, lightweight solution for next-generation secure IIoT systems.

Keywords: decentralized authentication; PUF; SSS; revocable credential; IIoT



Academic Editor: Gildas Avoine

Received: 14 November 2025

Revised: 12 December 2025

Accepted: 15 December 2025

Published: 20 December 2025

Copyright: © 2025 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and

conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

In the Industrial Internet of Things (IIoT), combining operational technology (OT) and information technology (IT) has created major security problems. These problems affect resource-constrained devices—like sensors and edge terminals used in open industrial settings—since their limited computing power, storage and energy make them incompatible with complex security protocols such as traditional Public Key Infrastructure (PKI). Worse still, frequent device–server interactions and transparent network boundaries expose them to escalating risks of unauthorized access and data breaches [1].

Current authentication systems designed for these scenarios often have significant vulnerabilities, as they struggle to balance security, efficiency, and privacy. A key weakness is their dependence on centralized trusted third parties (TTPs) for identity verification and credential management. This not only introduces single points of failure but also leads to

performance bottlenecks in large-scale IIoT deployments. Meanwhile, many lightweight authentication protocols used in telehealth and vehicular-area networks already utilize random nonces and hash-based authenticators to defend against replay attacks and generate fresh session keys [2–5]. These mechanisms work well for ensuring message freshness and protecting individual sessions, but they do not eliminate the fundamental reliance on long-term static credentials managed by a central authority. Another problem is the common use of static credentials like biometrics on the back end. Unlike passwords, which can be reset, biometric templates are permanent and cannot be replaced. So, if a centrally stored template is compromised, the resulting privacy breach is irreversible—a risk that is made worse by the centralized storage of credentials in most existing schemes [6].

Additionally, lightweight authentication solutions using physical unclonable functions (PUFs) have been a promising approach for these devices. However, conventional PUFs are vulnerable to modeling attacks, which limits their reliability in adversarial IIoT environments. Together, these overlapping challenges—centralization risks, privacy flaws in static credentials, and PUF vulnerabilities—make it urgent to develop a secure, efficient and lightweight authentication framework designed for IIoT characteristics.

To solve the above challenges, this paper proposes a new lightweight and decentralized authentication scheme called DLR-Auth. Its main contributions are as follows:

1. New decentralized architecture: A two-party design eliminates the online registration center at enrollment. Using matched hardware properties, the user and the server establish trust directly. This simplifies the system model, improves scalability by removing bottlenecks, and reduces single points of failure.
2. Combined use of key technologies: Chaotic synchronization of SSL-PUFs is integrated with Shamir's secret sharing (SSS). An SSL-PUF can derive identical keys on two devices from a single challenge, enabling direct enrollment without sending the underlying secret over the network. SSS then builds a revocable template from the hardware-derived secret, supporting template revocation without exposing biometric data [7].
3. Lightweight protocol suite: An end-to-end suite covers four phases: decentralized enrollment, mutual authentication, efficient multi-access, and template revocation. The protocols use symmetric-key primitives and hashing while avoiding costly public-key operations, making them suitable for resource-constrained devices.
4. Hardware-based prototyping and evaluation: The practicality of the scheme is demonstrated by outlining its implementation on real hardware with the design inspired by advanced validation methods. A comprehensive evaluation framework covering computational overhead, communication cost and resource usage is provided. Results are compared against the latest academic research to verify the efficiency and competitiveness of the scheme.

The rest of this paper is structured as follows. Section 2 reviews related work. Section 3 introduces the system model and threat model, and then presents the proposed DLR-Auth scheme. Section 4 provides the security analysis. Section 5 presents the performance evaluation. Section 6 concludes the paper [8].

2. Related Work

2.1. A Survey of Lightweight Authentication Protocols

Secure Internet of Things (IoT) authentication protocols enable parties to verify each other and establish secure channels. Traditional approaches often rely on weak credentials or impose high computational cost. IoT deployments need methods that are both lightweight and secure. Recent work have proposed using PUFs for mutual authentication. A PUF is a hardware function with device-unique physical characteristics. Because its responses are hard to predict or clone, PUF-enabled devices are used for remote authentication.

Rührmair et al. (2010) [9] first showed a fundamental challenge for early strong PUFs. Many mainstream designs including the Arbiter PUF are vulnerable to modeling attacks. After collecting sufficient challenge–response pairs (CRPs), an attacker can train a model to predict PUF behavior. For example, their experiments indicate that once the in-between of the 10^4 and 10^5 CRPs are collected for a typical Arbiter PUF, machine learning attacks can reach prediction accuracies well above 95%, practically undermining the assumed unclonability. This exposed a significant vulnerability in early PUF-based protocols.

Early strong PUF-based authentication protocols usually followed a simple challenge–response model. A remote verifier would send a query to the PUF over an open channel, then check authenticity by comparing the received response with data in a stored challenge–response pair (CRP) database. Reusing this approach in such scenarios effectively exposes large numbers of raw CRPs to attackers, which significantly increases the risk of machine-learning modeling attacks on the PUF itself. This issue is common in early strong PUF research and has led to efforts to develop countermeasures both in PUF design and protocol structure. More recent PUF-based schemes for telehealth and vehicular-area networks use a different design approach: the PUF is part of a larger cryptographic system, and its outputs are immediately processed by fuzzy extractors, hash functions, or key-derivation tools—rather than being sent in plaintext over the network. Typical examples from MDPI publications include dynamic authentication for telehealth systems and PUF-based Internet of Vehicles (IoV) authentication. In these schemes, PUF responses are combined with helper data to generate device-specific keys, without revealing raw CRPs to attackers [10–13]. The proposed DLR-Auth framework adopts this modern design: SSL-PUF responses are only used within the secure environment of the device and its server-side twin, and are processed locally to derive internal secrets. No raw PUF outputs or large CRP databases are transmitted over the authentication channel.

To counter modeling attacks, defenses emerged at both the protocol and hardware levels. At the protocol level, Yu et al. (2016) [14] introduced the lockdown technique, which programmatically restricts CRP collection and impedes model training. At the hardware level, designers explored structures that are harder to model. Plusquellic (2022) [15–17] described the SiRF PUF on FPGAs. It uses a logic-gate network to increase entropy and obfuscate internal path delays, which raises modeling difficulty.

Beyond early strong PUFs, a series of PUF-assisted mutual authentication and key-agreement protocols published from 2019 to 2025 have progressively moved towards lightweight and often decentralized designs. A second issue tackled by these works is the reliance on a central verifier that stores a large, secure CRP database, which is impractical in distributed IIoT settings. To address this, Chatterjee et al. (2018) [18] combine PUFs with identity-based encryption (IBE) and keyed hash functions so that the verifier no longer needs to maintain explicit CRPs. For end-to-end device communication, Zheng et al. (2022) [19] propose a peer-to-peer IoT session protocol that uses a one-time-pad (OTP) mask and hash-based authenticators to achieve mutual authentication and key establishment without a dedicated server. A further direction is “chaotic synchronization,” which aims to eliminate CRP databases entirely: Gao et al. (2025) [20] propose a twin-device framework based on SSL-PUFs, where two physically matched PUFs independently derive an identical key under the same challenge, thus enabling database-free key agreement.

Template security and revocability are another focus. Leakage of static, life-long credentials (e.g., biometric templates) leads to severe consequences. Researchers therefore introduced revocable templates and SSS into authentication systems. Yu et al. (2021) [21] integrated PUFs with revocable biometrics and used SSS to manage templates in multi-server settings, and demonstrated the feasibility of using SSS in an industrial IoT data-protection scenario by combining Shamir’s secret sharing with a blockchain-based threshold cryp-

tography scheme to protect data keys in a permissioned, multi-party setting. While their model relies on a blockchain ledger and thus differs from our two-party, non-permissioned context, it shows that SSS can support decentralized key-management tasks under different trust assumptions. In contrast, DLR-Auth does not assume any blockchain infrastructure and applies SSS locally between UD_i and S .

Beyond the IIoT context, many studies have explored lightweight authentication in telehealth and vehicular-area networks using nonces and hash-based primitives. Ryu and Kim proposed a privacy-preserving authentication protocol for wireless body area networks in healthcare applications. This protocol only relies on one-way hash functions and exclusive-OR (XOR) operations to achieve mutual authentication and establish session keys in each run [2]. El-Meniawy et al. designed an authentication protocol for the medical Internet of Things, where hashing and XOR are used to authenticate medical devices and back-end servers over public channels [3]. In the vehicular field, Wang et al. presented a lightweight, physically secure key-agreement and authentication protocol for vehicular networks. It combines elliptic-curve cryptography (ECC) with hash functions, XOR operations, and PUF primitives [4]. Meanwhile, Liu et al. developed a V2X authentication scheme for task offloading in edge-assisted autonomous driving. This scheme uses ECC, hash operations, and PUFs to reduce computational and communication overhead as much as possible [5]. These protocols show that nonce-based authenticators and hash-derived session keys have become standard in related application domains. The work is complementary: it focuses on making the underlying long-term credentials revocable by deriving them from paired SSL-PUFs and protecting them through secret sharing.

Despite these advances, many schemes still rely on a trusted third party during initial registration or ignore efficiency demands such as “one authentication, multiple accesses.” Banerjee et al. (2021) [22] emphasized that fog computing requires more dynamic efficient re-authentication and fast failover. Minwalla et al. (2024) [23] proposed PUF-MAKE. It is lightweight but still uses a three-party model with a trusted authentication server. Table 1 compares DLR-Auth with representative state-of-the-art schemes along four dimensions: system architecture, the need for an explicit CRP database, credential revocability, and fast re-access support. Existing schemes usually provide only part of these properties, such as decentralization without revocability, or revocability that still depends on a TTP-based architecture. In contrast, DLR-Auth is a two-party decentralized framework that uses SSL-PUF-based chaotic synchronization and Shamir’s secret sharing to achieve secure, efficient, and revocable authentication for IIoT deployments.

Table 1. Feature comparison of DLR-Auth with related state-of-the-art schemes.

Authors	Architecture	No Explicit CRP DB	Credential Revocability	Fast Re-Access (One-Auth Multi-Access)	Core Techniques
Chatterjee et al. (2019) [18]	TTP-based	Yes	No	No	IBE, PUF, ECC, keyed hash
Zheng et al. (2022) [19]	P2P	Yes	No	No	PUF, OTP masking, ECC
Yu et al. (2021) [21]	Decentralized	Yes	Yes	No	Blockchain, Shamir’s secret sharing
Yu et al. (2024) [24]	TTP-based	No	Yes	Yes	Arbiter PUF, revocable biometrics, FE, SSS
Gao et al. (2025) [20]	Decentralized	Yes	No	No	SSL-PUF, chaotic sync, compressed sensing
Our scheme (DLR-Auth)	Decentralized	Yes	Yes	Yes	SSL-PUF, Shamir’s secret sharing, chaotic sync

2.2. Semiconductor Superlattice PUF

The core primitive is the SSL-PUF [20,25]. It derives entropy from the physical behavior of a superlattice, modeled as a high-dimensional nonlinear dynamical system whose evolution depends on device-specific parameters.

Let the PUF state be $\mathbf{z}(t) \in \mathbb{R}^n$. Its dynamics are as follows:

$$\dot{\mathbf{z}}(t) = \mathbf{F}(\mathbf{z}(t), x(t), \mathbf{p}), \quad (1)$$

where $x(t)$ is the challenge input and $\mathbf{p}$ is a vector of device parameters. The vector $\mathbf{p}$ acts as a device fingerprint and can induce chaotic behavior.

A circuit-level model is

$$\begin{aligned} \dot{v}_C(t) &= \frac{1}{C_1} (x(t) - v_C(t) - v_{SLD}(t)), \\ \dot{v}_{SLD}(t) &= \frac{1}{C_2} (v_C(t) - i_{SLD}(v_{SLD}(t))), \end{aligned} \quad (2)$$

where $v_C(t)$ denotes the voltage across the coupling capacitor and $v_{SLD}(t)$ denotes the terminal voltage of the superlattice device (SLD); C_1 and C_2 are circuit capacitances. The term $i_{SLD}(v_{SLD}) = g(v_{SLD}, p)$ models the nonlinear current–voltage characteristic of the SLD as a function of the device voltage and parameters p . Let $z(t) = [v_C(t), v_{SLD}(t)]^\top$, and take the output through an RC network $y(t) = h(z(t))$.

A key property for decentralization is chaotic synchronization [20]. For two matched SSL-PUF devices with parameter vectors $p_1 \approx p_2$ driven by the same input $x(t)$, their state evolution can be written as

$$\dot{z}_1(t) = F(z_1(t), x(t), p_1), \quad \dot{z}_2(t) = F(z_2(t), x(t), p_2). \quad (3)$$

Here, $z_1(t)$ and $z_2(t)$ denote the state vectors of the two matched SSL-PUF devices. Let $y_1(t) = h(z_1(t))$ and $y_2(t) = h(z_2(t))$ be the scalar outputs of the two devices obtained through the same RC network $h(\cdot)$. Prior work on superlattice-based chaotic oscillators show that when p_1 and p_2 are sufficiently close and a common driving signal $x(t)$ is applied, the synchronization manifold $z_1(t) = z_2(t)$ is stable; hence the state error $e(t) = z_1(t) - z_2(t)$ converges to a small bounded neighborhood of zero. As a result, $y_1(t)$ and $y_2(t)$ become highly correlated. We quantify this correlation using the Pearson coefficient

$$\rho(y_1, y_2) = \frac{\text{cov}(y_1(t), y_2(t))}{\sigma_{y_1} \sigma_{y_2}} > \tau_{\text{corr}}, \quad (4)$$

with τ_{corr} typically above 0.95. It is important to note that while the underlying SSL-PUF dynamics are chaotic and nonlinear, the objective of our scheme is to achieve high-precision synchronization where the output signals $y_1(t)$ and $y_2(t)$ are nearly identical ($y_1(t) \approx y_2(t)$). In this regime, the relation between the two sampled outputs can be locally approximated as linear, so the Pearson coefficient serves as an efficient and practical metric for quantifying signal matching and synchronization stability. We do not use $\rho(y_1, y_2)$ to analyze the full nonlinear dynamics of the chaotic system, but only as an empirical threshold to decide when the fuzzy extractor can reliably derive a common key.

A fuzzy extractor converts the noisy agreement to a shared key [24]. The first device computes $(K, P) = \text{Gen}(y_1(t))$ and sends helper data P . The second device reconstructs $K' = \text{Rep}(y_2(t), P)$ with $K' = K$, establishing a shared secret without centralized key storage.

2.3. SSS Scheme

SSS is a threshold scheme for distributing a secret among n parties [21,26]. It mitigates single-point compromise by splitting the secret into n unique shares. Under a (t, n) policy, any set of at least t shares can reconstruct the secret, while any set of at most $t - 1$ shares reveals no information.

The scheme encodes the secret s as the constant term of a random polynomial $f(x)$ of degree $t - 1$ over a prime field $\mathbb{Z}_p$:

$$f(x) = a_{t-1}x^{t-1} + \dots + a_1x + s \pmod{p}, \quad (5)$$

where p is a sufficiently large prime, the secret satisfies $s \in \mathbb{Z}_p$, and the coefficients $a_1, \dots, a_{t-1} \in \mathbb{Z}_p$ are chosen independently and uniformly at random. Distinct evaluation points $(x_i, f(x_i))$ form the shares distributed to the n participants.

Reconstruction relies on the fact that t points determine a degree- $(t - 1)$ polynomial. Given any t shares $\{(x_j, y_j)\}_{j=1}^t$, Lagrange interpolation recovers

$$s = f(0) = \sum_{j=1}^t y_j \prod_{\substack{m=1 \\ m \neq j}}^t \frac{x_m}{x_m - x_j} \pmod{p}. \quad (6)$$

In the proposed framework, SSS underpins the construction and management of revocable templates in a distributed setting.

3. Model and Proposed Scheme

3.1. System and Threat Models

3.1.1. System Model

The scheme uses a decentralized two-party design with a user device UD_i and a service S that communicate over a public channel, as illustrated in Figure 1. Figure 1 shows the two main entities and the insecure network over which protocol messages are exchanged. No always-online registration authority is required, which improves scalability and avoids a single point of failure.

S is a relying party with limited trust. It issues challenges, coordinates sessions, and may keep non-secret metadata or assigned secret-sharing shares. For each registered user, S holds a matched SSL-PUF module that serves as a hardware root of trust. In large-scale deployments, the pairing of this SSL-PUF scales linearly with the number of devices: each new device adds one twin instance locally and one corresponding instance or logical entry at the server, along with a small volume of secret-sharing metadata. Notably, no global CRP database or extra online authority is required. This fixed per-device resource footprint renders the approach feasible for IIoT-scale deployments.

UD_i is the requester, such as an IIoT sensor or a mobile terminal. It integrates a twin SSL-PUF matched to the server module, which enables key generation by chaotic synchronization. The device sends requests, produces secrets in response to challenges, and protects its credentials in secure non-volatile memory or on a secure element.

Here, the absence of an online registration authority (RA) means that no additional third party is required during enrollment or authentication. The server S remains the single central relying party and hardware root of trust for the domain, and the security implications of this assumption are discussed in the threat model and in the limitations.

Communication occurs over an insecure public channel with an active adversary in the Dolev–Yao model. The adversary can read traffic and can delay, replay, modify, or inject messages. Security does not rely on channel properties. It follows from the unforgeability of the SSL-PUF and from the threshold secrecy of the secret-sharing layer.

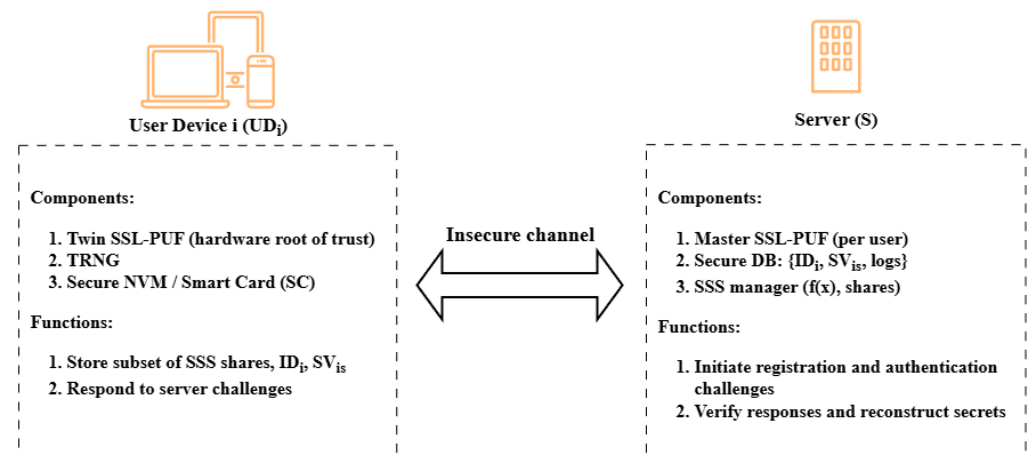


Figure 1. System model of the DLR-Auth authentication scheme.

3.1.2. Threat Model

The adversarial model covers network-level and physical-level threats and aligns with the proof setting in Section 5.

At the network level, a Dolev–Yao adversary controls the channel. Messages can be eavesdropped, modified, injected, replayed, delayed, reordered, or dropped. These actions may cause denial of service or desynchronization. Cryptographic primitives are treated as ideal in the Dolev–Yao sense. The adversary cannot invert hash functions or break symmetric encryption without the relevant keys. Availability is not guaranteed. The focus is confidentiality and authenticity when communication proceeds.

At the physical level, only the user device is exposed. The server is physically protected and faces only network-level attacks. A non-invasive side-channel adversary may measure power, timing, or electromagnetic leakage during protocol execution and during PUF evaluation, as reported in prior work [20,27]. The analysis assumes such leakage does not reveal the internal state of the PUF or protected keys in a way that violates standard cryptographic models.

Device capture with memory extraction is considered. The adversary may read contents stored in insecure non-volatile memory on the device, including identifiers, helper data, and cached tokens. Security goals should hold under such exposure. A protected boundary is assumed for hardware-protected elements, including the PUF core and any secure-element keys. The adversary cannot read or clone the internal state of the PUF. Moreover, the adversary never observes raw SSL-PUF responses over the network. PUF outputs are evaluated only within the protected boundary of the device or the server-side twin and are immediately processed by fuzzy-extractor and key-derivation functions before any values leave the hardware. Only helper data and derived cryptographic keys are exported, in line with recent PUF-based authentication schemes for telehealth and IoV. We assume that the fuzzy extractor applied to the SSL-PUF outputs satisfies the standard correctness and security properties, namely that reconstructed keys agree with overwhelming probability and that the public helper data leaks at most negligible information about the extracted key.

Physical tampering that destroys or desynchronizes the PUF is treated as an availability failure. In that case, the device is detected and refused during authentication. Confidentiality and resistance to impersonation remain; liveness is not guaranteed. In practice, restoring functionality in such a case requires hardware re-provisioning: the operator installs a new matched SSL-PUF pair and re-runs the decentralized registration phase, thereby deriving a fresh hardware secret R_{new} and issuing new SSS templates bound to it. In practice, recovery from such an event requires hardware re-provisioning: the operator installs a new matched

SSL-PUF pair and re-runs the decentralized registration phase, thereby deriving a fresh hardware secret R_{new} and issuing new SSS templates bound to it.

The scope of compromise excludes physical access to the server and its long-term secrets. The analysis targets resistance to impersonation and replay under Dolev–Yao control of the channel, session-key indistinguishability under transcript exposure and non-volatile memory extraction, and robustness against non-invasive side-channel observation. Availability against denial of service is out of scope. Detection and conservative lockout are intended.

This threat model specifies what the adversary may observe or extract from insecure device storage and what remains protected, such as the PUF internal state and server secrets. The ROR-based key indistinguishability proof in Section 5 uses these assumptions.

3.2. The Proposed Scheme (DLR-Auth)

3.2.1. Scheme Overview

The proposed Decentralized, Lightweight, and Revocable Authentication scheme (DLR-Auth) establishes a secure channel between a user device UD_i and a server S without a trusted third party. The architecture is a two-party model. Each UD_i carries a twin SSL-PUF that is paired during provisioning with a master SSL-PUF on S .

The protocol has three phases. The first phase is decentralized registration. UD_i and S set up an initial security context directly. No online registration authority is involved. The second phase is mutual authentication and key agreement. A challenge–response procedure invokes the PUFs to reproduce protected randomness. Fresh nonces and hash-based derivation then produce an ephemeral session key that authenticates both parties. The third phase is efficient multi-access. After one full authentication, subsequent accesses within a short window use lightweight verification to reduce computation and communication.

The design follows three principles. The first is decentralization. Removing a central registration service improves scalability and reduces single points of failure. The second is lightweight construction. The protocol uses hash functions and symmetric-key operations and relies on the PUF as a hardware root rather than storing long-term keys in insecure non-volatile memory. The third is revocability. Credentials are bound to revocable shares, for example via Shamir-style secret sharing [28], so that a device can be invalidated without changing server-wide secrets.

Security goals are aligned with the threat model. The PUF pairing provides device uniqueness and resists cloning within the stated assumptions. Ephemeral session keys derived from fresh inputs mitigate the impact of transcript exposure by avoiding key reuse across sessions. The multi-access mechanism reduces overhead while preserving authentication guarantees within its validity window. Detailed security arguments are given in Section 5.

3.2.2. Decentralized Registration Phase

The registration phase is a third-party-free and two-party decentralized process involving only UD_i and S . The core objective is to establish a shared context, bootstrapped by the chaotic synchronization property of paired SSL-PUFs [29]. The interaction flow is illustrated in Figure 2, and proceeds as follows:

Step 1 Registration Request.

UD_i generates a fresh random nonce n_0 and computes a one-time temporary pseudonym $PID_i = ID_i \oplus \text{KDF}(h(n_0), |ID_i|)$, where $\text{KDF}(\cdot, |ID_i|)$ denotes a length-aligned derivation. It then transmits:

$$UD_i \rightarrow S : \{ID_S, n_0, PID_i\}.$$

Step 2 Challenge and Encrypted Helper Data (SSS).

Upon receiving the request, the Server generates a one-time challenge C_{reg} and evaluates its Master SSL-PUF to compute $R_i = \text{SSL-PUF}_S(C_{reg})$. The Server sets the Shamir secret as $f(0) := R_i$, and samples a random polynomial $f(x)$ of degree $(t - 1)$ to define a threshold- t SSS. It then computes and selects a set of helper shares:

$$HD = \{(j, s_j) \mid s_j = f(j), j \in \mathcal{J}, |\mathcal{J}| = t-1\},$$

containing $(t - 1)$ distinct points. The Server also generates a long-term shared key SV_{is} or defines it as a key derived from R_i if desired. To protect these materials, it derives a temporary key $k = h(R_i \parallel n_0)$, and applies authenticated encryption:

$$C_{data} = \text{AEAD}_k(\{HD, SV_{is}\}, \text{AD} = C_{reg} \parallel n_0).$$

This package is sent back to the User Device along with the public challenge:

$$S \rightarrow UD_i : \{C_{reg}, C_{data}\}.$$

Step 3 Secret Generation and Confirmation.

The User Device evaluates its twin SSL-PUF on C_{reg} to synchronously obtain $R'_i = \text{SSL-PUF}_{UD_i}(C_{reg})$, and derives the same temporary key $k' = h(R'_i \parallel n_0)$. It then decrypts C_{data} and verifies the AEAD tag to recover $\{HD, SV_{is}\}$. After the integrity check passes, UD_i securely stores HD and sets up its Smart Card (SC). Note that UD_i does not hold polynomial coefficients or a complete set of t shares. So, to reconstruct $f(0) = R_i$ in any later phase, UD_i must combine its stored $t - 1$ helper shares with an additional share (j^*, s_j^*) —this share is generated and sent by S during authentication. On their own, neither the device nor the server can reconstruct R_i using only their own long-term data. To conclude the phase, it computes a confirmation message

$$M_{confirm} = h(ID_i \parallel R'_i \parallel SV_{is} \parallel C_{reg} \parallel n_0),$$

and sends it to the Server:

$$UD_i \rightarrow S : \{M_{confirm}\}.$$

Step 4 Finalization and Archiving.

To finalize the registration, the Server computes

$$M'_{confirm} = h(ID_i \parallel R_i \parallel SV_{is} \parallel C_{reg} \parallel n_0),$$

and checks $M'_{confirm} \stackrel{?}{=} M_{confirm}$. If the verification succeeds, the Server archives a permanent access record for UD_i in its audit storage (sufficient for audit/revocation linkage but not for reconstructing the secret), successfully completing the process. These steps correspond to the blocks and arrows in Figure 2, which summarize the registration messages and the state updates at UD_i and S .

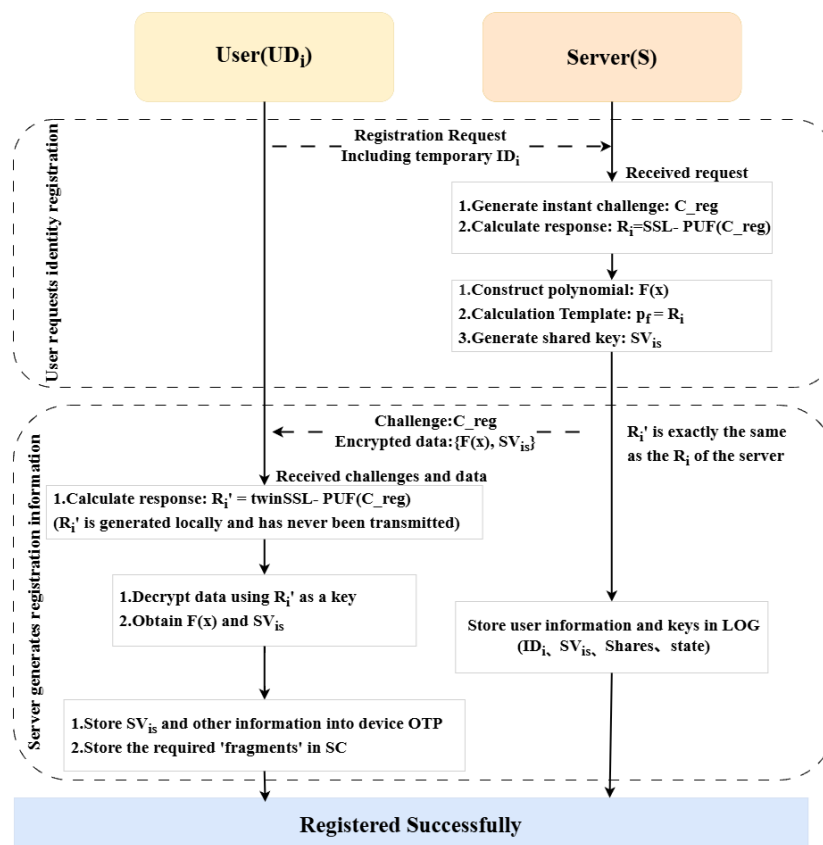


Figure 2. Flowchart of the decentralized registration phase of the DLR-Auth scheme.

3.2.3. Mutual Authentication and Key Agreement Phase

Following a successful decentralized registration, UD_i and S can engage in the mutual authentication and key agreement phase to cryptographically verify each other's legitimacy and establish a fresh, ephemeral SSK. This process leverages the previously established shared context. The interaction flow of this phase is shown in Figure 3, and the detailed steps are as follows:

Step 1 Authentication Request.

To initiate an authenticated session, UD_i generates a fresh random nonce n_0 and computes a temporary pseudonym $PID_i = ID_i \oplus \text{KDF}(h(n_0 \parallel SV_{is}), |ID_i|)$. It then sends:

$$UD_i \rightarrow S : \{ID_S, n_0, PID_i\}.$$

Step 2 Server Challenge.

S uses ID_S to retrieve $\{ID_i, SV_{is}, \text{meta}\}$ and verifies $ID'_i = PID_i \oplus \text{KDF}(h(n_0 \parallel SV_{is}), |ID_i|) = ID_i$. It samples n_1 and a hardware challenge C_{auth} , sets $k_S := h(ID_i \parallel SV_{is})$, $AD_S := (ID_S \parallel n_1)$, and runs AEAD on payload $\{n_1, C_{auth}\}$ to obtain a ciphertext-tag pair (M_1, M_2) , then sends

$$S \rightarrow UD_i : \{M_1, M_2\}.$$

Step 3 User Verification and Encrypted Proof.

UD_i AEAD-decrypts (M_1, M_2) under (k_S, AD_S) to recover $\{n_1, C_{auth}\}$. It samples n_2 , sets $k_U := h(ID_S \parallel SV_{is})$, $AD_U := (ID_i \parallel n_0 \parallel n_1)$, and AEAD-encrypts n_2 into $\mathcal{E}$. Preserving its shape, we serialize it as

$$UD_i \rightarrow S : \{E_1, \dots, E_{t+2}\} \equiv \mathcal{E}.$$

Step 4 Secret Reconstruction Aid and Final Challenge.

S recovers n_2 , computes $R_i := \text{SSL-PUF}_S(C_{\text{auth}})$, $R_{\text{rec}} := f(0)$, and verifies $R_i = R_{\text{rec}}$.

It chooses a fresh index j^* and computes $s_{j^*} = f(j^*)$. Then, it samples n_3 and sets $\text{AD}'_S := (ID_S \parallel n_1 \parallel n_2)$, $\mathcal{M} := \text{AEAD}_{k_S}(\{j^*, s_{j^*}\}, \text{AD}'_S)$. It sends

$$S \rightarrow UD_i : \{M_3, \dots, M_{t+3}\} \equiv (n_3, \mathcal{M}).$$

Step 5 Key Generation and Final Confirmation.

UD_i receives $(n_3, \mathcal{M})$ and AEAD-decrypts $\mathcal{M}$ under $\text{AD}'_S = (ID_S \parallel n_1 \parallel n_2)$ to obtain $\{j^*, s_{j^*}\}$. Using its local $t-1$ helper shares and s_{j^*} , it reconstructs $R'_{\text{rec}} = f(0)$. With a fresh n_4 , derive $\text{SSK}_{iS} := \text{HKDF}(\text{IKM} = R'_{\text{rec}}, \text{salt} = \text{SV}_{iS}, \text{info} = ID_i \parallel ID_S \parallel n_1 \parallel n_2 \parallel n_3 \parallel n_4)$. Send the final confirmation (AEAD):

$$E_{t+3} := \text{AEAD}_{k_U}(n_4, \text{AD} = n_3), \quad E_{t+4} := \text{AEAD}_{\text{SSK}_{iS}}(\text{confirm}, \text{AD} = n_4),$$

and serialize it as

$$UD_i \rightarrow S : \{E_{t+3}, E_{t+4}\}.$$

Step 6 Final Verification.

S recovers n_4 , derives $\text{SSK}'_{iS} := \text{HKDF}(\text{IKM} = R_{\text{rec}}, \text{salt} = \text{SV}_{iS}, \text{info} = ID_i \parallel ID_S \parallel n_1 \parallel n_2 \parallel n_3 \parallel n_4)$, and verifies E_{t+4} with $\text{AD} = n_4$. If valid, mutual authentication succeeds and SSK_{iS} is established. These six steps map directly to the message exchanges and local computations shown in Figure 3, where each arrow represents one protocol message and each box indicates a state update at UD_i or S .

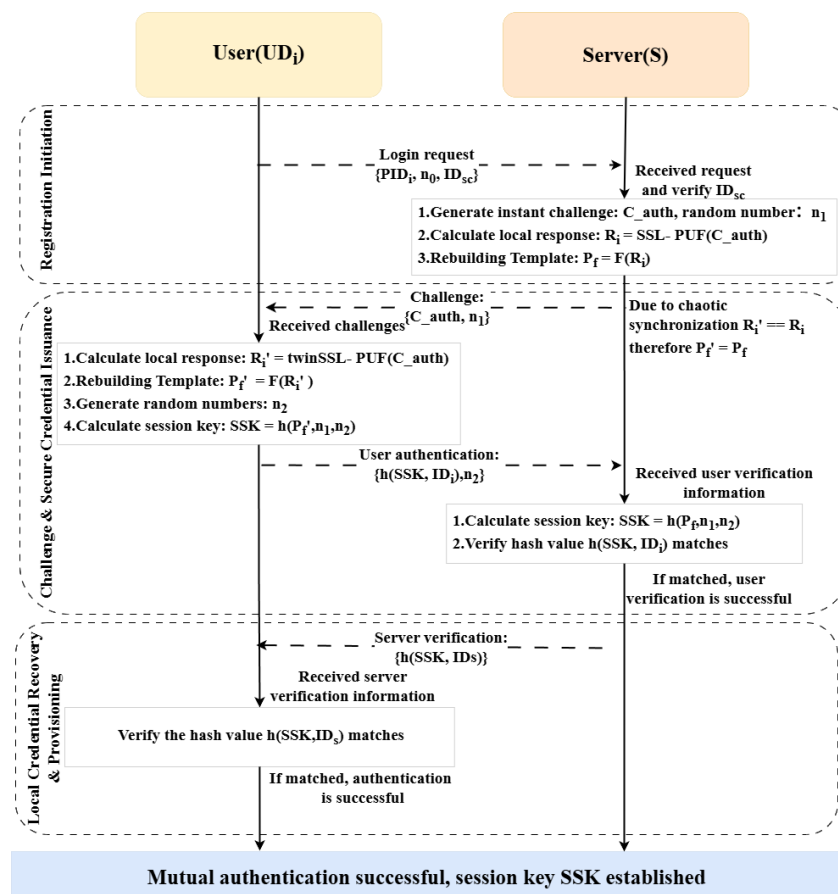


Figure 3. Flowchart of the mutual authentication and key agreement phase of the DLR-Auth scheme.

3.2.4. One-Authentication Multiple-Access Mechanism

To improve efficiency for frequent access, the scheme uses a lightweight mechanism. After an initial assessment, successful authentication establishes a secure context which includes a shared session key SSK_{iS}^S , an updated identifier ID'_{SC} , a validity period ΔT , and an access limit b . Here, ΔT and b are policy-level parameters rather than cryptographic security parameters: ΔT specifies the wall-clock time window after a successful full authentication during which fast re-access is permitted, and b upper-bounds the number of fast-access requests that UD_i may issue within this window. Their concrete values are chosen according to deployment-specific usability and risk requirements; as long as ΔT is shorter than the time scale of feasible replay attempts in the assumed setting and b matches the maximum legitimate reconnection count, they do not affect the secrecy of the established keys and only bind the replay window for the fast-access mechanism. This process is illustrated in Figure 4 and consists of the following two steps. The main parameters used in this One-Authentication Multiple-Access mechanism are summarized in Table 2.

Table 2. System parameters for the One-Authentication Multiple-Access mechanism.

Parameter	Description
SSK_{iS}	Session key between UD_i and S .
ID'_{SC}	Updated smart-card identifier after full authentication.
ΔT	Time window in which fast re-access is allowed.
b	Maximum number of fast-access requests within ΔT .

Step 1 Fast-Access Request.

First, derive feature-specific keys via HKDF:

$$(k_{\text{enc}}, k_{\text{mac}}) := \text{HKDF}(SSK_{iS}, \text{salt} = SV_{iS}, \text{info} = \text{fast_access}).$$

Let T_k be the current timestamp and ctr_i a strictly increasing session counter. Compute

$$E_{t+5} := \text{AEAD}_{k_{\text{enc}}}(ID'_{SC}; \text{AD} = ID_S \parallel T_k \parallel ctr_i),$$

$$E_{t+6} := \text{HMAC}_{k_{\text{mac}}}(T_k \parallel ctr_i \parallel \text{ct}(E_{t+5})).$$

Transmit

$$UD_i \rightarrow S : \{T_k, ctr_i, E_{t+5}, E_{t+6}\}.$$

Step 2 Server Verification.

Upon receipt, S first checks freshness and quota, i.e., $(T_k - T_C) < \Delta T$ and $\text{count}_i < b$, ensuring that any stolen or replayed fast-access tokens cannot be used beyond the configured time and usage limits. It then enforces anti-replay by requiring the session counter ctr_i to be strictly larger than the last stored value for this user. If these checks pass, S derives per-feature keys via

$$(k_{\text{enc}}, k_{\text{mac}}) := \text{HKDF}(SSK_{iS}, \text{salt} = SV_{iS}, \text{info} = \text{fast_access}).$$

Next, S authenticates the ciphertext and context by verifying

$$\text{HMAC-Verify}_{k_{\text{mac}}}(E_{t+6}, T_k \parallel ctr_i \parallel \text{ct}(E_{t+5})).$$

Upon success, it decrypts $ID''_{SC} := \text{AEAD}_{k_{\text{enc}}}^{-1}(E_{t+5}; \text{AD} = ID_S \parallel T_k \parallel ctr_i)$. Access is granted iff $ID''_{SC} = ID'_{SC}$. If granted, S updates count_i and persists with the new ctr_i ; other-

wise, it denies the request, applies rate limiting/backoff, and triggers full re-authentication once a failure threshold is exceeded.

These two steps correspond to the fast re-access flow in Figure 4, where the first message encodes the cached context and the second step verifies the request and updates the local counters at UD_i and S .

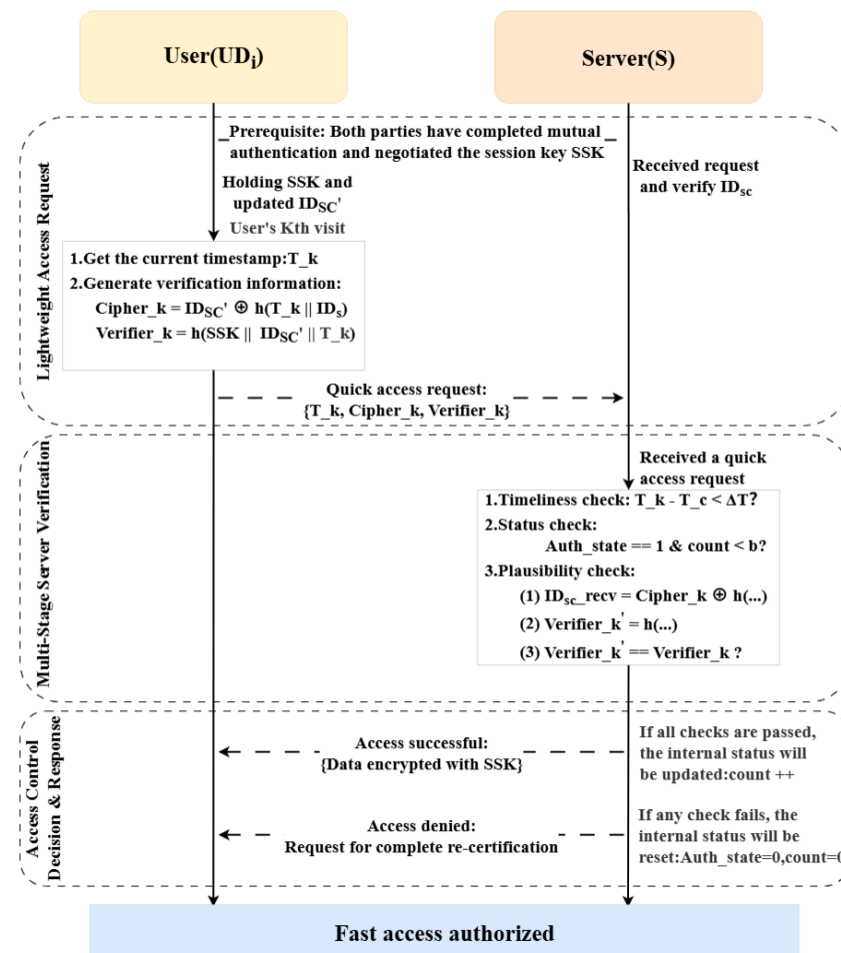


Figure 4. Flowchart of the One-Authentication Multiple-Access mechanism of the DLR-Auth scheme.

3.2.5. Template Revocation and Update Mechanism

When credentials must be rotated or are suspected compromised, revocation and update are executed directly between UD_i and S in four steps.

Step 1 Authenticated Revocation Request.

A fresh mutual authentication establishes a valid session key SSK_{iS} . Derive a revocation-scoped key $k_{rev} := \text{HKDF}(SSK_{iS}, \text{salt} = SV_{iS}, \text{info} = \text{revoke})$. $2 \cdot T_h = 2 \cdot (0.07) = 0.14$. Let the client sample a nonce n_r and form an AEAD-authenticated request $\text{Req}_{\text{revoke}} := \text{AEAD}_{k_{rev}}(\{\text{"REVOKE"}, ID_i, n_r\}; \text{AD} = ID_S \parallel T \parallel v)$, then send

$$UD_i \rightarrow S : \{\text{Req}_{\text{revoke}}\}.$$

Step 2 New Polynomial and Helper Data.

Upon validation, S samples a new $(t - 1)$ -degree polynomial $f'(x)$ and again fixes the Shamir secret to the current hardware-derived value R_i , i.e., $f'(0) := R_i$. It selects $t - 1$ fresh indices J to obtain $HD' = \{(j, s'_j) \mid s'_j = f'(j), j \in J\}$, excluding $x = 0$ and all polynomial coefficients. This operation re-randomizes the polynomial coefficients and

the set of helper shares while leaving the underlying SSL-PUF secret unchanged; in other words, “revocation” here means invalidating the previous SSS template and associated long-term values, not changing R_i itself. If R_i itself must be renewed, the device is re-provisioned by installing a new matched SSL-PUF instance and executing the decentralized registration phase to bind a fresh hardware secret to newly generated templates.

Step 3 Secure Distribution of Updates.

S generates a new long-term shared value SV'_{iS} and a version $v' = v + 1$. The update package is AEAD-protected under k_{rev} : $C_{\text{update}} := \text{AEAD}_{k_{\text{rev}}}(\{HD', SV'_{iS}, v'\}; AD = ID_S \parallel n_r)$, and is sent as

$$S \rightarrow UD_i : \{C_{\text{update}}\}.$$

Step 4 Credential Update and Finalization.

UD_i decrypts and verifies C_{update} , installs HD' and SV'_{iS} , and returns an AEAD confirmation $\text{Ack} := \text{AEAD}_{k_{\text{rev}}}(\text{“ACK”}; AD = v')$. S records the audit entry and version v' , invalidates prior versions, and avoids storing materials sufficient to reconstruct the secret alone.

3.2.6. Protocol Summary and Notations

This subsection gives a brief overview. Table 3 lists the step-by-step summary.

The protocol starts with two-party registration without a third party. The User Device and the Server use the paired SSL-PUFs to reach a shared hardware secret by chaotic synchronization. The secret is not sent on the network. The Server sets a SSS polynomial with the secret as $f(0)$. A small set of shares forms helper data for a revocable template.

After registration the parties run mutual authentication and key agreement. A fresh challenge–response re-validates the hardware and reconstructs the secret. An ephemeral session key is derived by HKDF from the reconstructed secret and session-unique values such as identities and nonces. The design provides session freshness, replay resistance, and binding to both parties. Strong forward secrecy is not ensured if the foundational secret is later exposed. Strong forward secrecy needs an extra per-session ephemeral input such as ephemeral Diffie–Hellman or one-time PUF randomness.

This scheme does not provide strict forward secrecy (FS), if the long-term secret R_i is compromised after a session, past session keys may be recoverable. Our goal is recording security rather than FS. A full FS construction and proof are left to future work.

Table 3 summarizes the computations and the message flow.

Table 3. DLR-Auth protocol steps.

Protocol Steps (Part 1)
1. $n_0 \leftarrow \text{TRNG}()$
2. $PID_i \leftarrow ID_i \oplus \text{KDF}(h(n_0), ID_i)$
3. $\mathbb{U}_i \rightarrow S : \{ID_S, PID_i, n_0\}$
4. S computes:
$C_{\text{reg}} \leftarrow \text{TRNG}()$
$R_i \leftarrow \text{SSL-PUF}_S(C_{\text{reg}})$
$f(x) \leftarrow \text{Gen_Poly}(R_i, t) \quad (f(0) = R_i)$
$HD \leftarrow \text{Gen_Shares}(f(x), t)$
$SV_{iS} \leftarrow \text{TRNG}()$
$k \leftarrow \text{KDF}(R_i \parallel n_0)$
$C_{\text{data}} \leftarrow E_k(\{HD, SV_{iS}\})$

Table 3. Cont.

5. $S \rightarrow UD_i : \{C_{reg}, C_{data}\}$
6. UD_i computes:
$R'_i \leftarrow \text{twin SSL-PUF}_{UD_i}(C_{reg})$
$k' \leftarrow \text{KDF}(R'_i n_0)$
$\{HD, SV_{iS}\} \leftarrow D_{k'}(C_{data})$
store SV_{iS}, HD
$M_{confirm} \leftarrow h(ID_i R'_i SV_{iS} C_{reg} n_0)$
7. $UD_i \rightarrow S : \{M_{confirm}\}$
8. S verifies:
$M'_{confirm} \leftarrow h(ID_i R_i SV_{iS} C_{reg} n_0)$
if $M_{confirm} = M'_{confirm}$, accept and store (ID_i, SV_{iS}, HD)
9. UD_i initiates authentication:
$n'_0 \leftarrow \text{TRNG}()$
$PID'_i \leftarrow ID_i \oplus \text{KDF}(h(n'_0 SV_{iS}), ID_i)$
10. $UD_i \rightarrow S : \{ID_S, n'_0, PID'_i\}$
11. S computes:
$n_1 \leftarrow \text{TRNG}()$

Protocol Steps (Part 2)

$M_1 \leftarrow h(ID_i SV_{iS} n_1)$
$M_2 \leftarrow h(HD ID_S n_1)$
12. $S \rightarrow UD_i : \{n_1, M_1, M_2\}$
13. UD_i verifies M_1, M_2 using n_1
14. UD_i computes:
$n_2 \leftarrow \text{TRNG}()$
$E_{HD} \leftarrow \text{EncryptShares}(HD, n_2, SV_{iS})$
15. $UD_i \rightarrow S : \{E_{HD}\}$
16. S computes:
$HD \leftarrow \text{DecryptShares}(E_{HD}, n_2, SV_{iS})$
$f(x) \leftarrow \text{ReconstructPoly}(HD)$
$C_{auth} \leftarrow \text{TRNG}()$
$n_3 \leftarrow \text{TRNG}()$
$k'' \leftarrow h(ID_i SV_{iS} n_3)$
$C_{poly} \leftarrow E_{k''}(f(x))$
17. $S \rightarrow UD_i : \{C_{auth}, n_3, C_{poly}\}$
18. UD_i computes:
$R'_i \leftarrow \text{twin SSL-PUF}_{UD_i}(C_{auth})$
$k'' \leftarrow h(ID_i SV_{iS} n_3)$
$f(x) \leftarrow D_{k''}(C_{poly})$
$pf' \leftarrow f(R'_i)$
$n_4 \leftarrow \text{TRNG}()$
$SSK \leftarrow \text{HKDF}(\text{IKM} = pf', \text{salt} = SV_{iS}, \text{info} = ID_i ID_S n_3 n_4)$
$V_{final} \leftarrow h(SSK SV_{iS} n_4)$
19. $UD_i \rightarrow S : \{n_4, V_{final}\}$
20. S computes SSK in the same way and verifies V_{final}

4. Security Analysis*4.1. Session-Key Indistinguishability in the ROR Model*

This subsection analyzes session-key indistinguishability in the Real-or-Random (ROR) model. The result does not establish forward secrecy. The baseline construction targets recording security under the stated leakage bounds rather than strict forward secrecy.

The goal is formalized as follows: any probabilistic polynomial-time adversary cannot distinguish the established session key from a uniformly random string in any fresh session. The model includes participants UD_i and S , and an adversary $\mathcal{A}$ with full network control

who can issue the standard oracle queries: Execute, Send, Reveal, Corrupt, and a single Test query on a fresh session that returns either the real key or a random string. The proof treats the composition of the SSL-PUF and the fuzzy extractor as a random oracle F_{PUF} , where the distinguishing advantage $\text{Adv}_{\text{PUF}}^{\text{RO}}(\mathcal{A})$ is negligible and upper bounded by ϵ_{puf} [30,31]. The hash function $h(\cdot)$ is modeled as a random oracle H with $\text{Adv}_{\text{Hash}}^{\text{RO}}(\mathcal{A}) \leq \epsilon_{\text{hash}}$. The symmetric encryption $E_k(\cdot)$ is assumed IND-CPA secure with $\text{Adv}_{\text{Enc}}^{\text{IND-CPA}}(\mathcal{A}) \leq \epsilon_{\text{enc}}$.

Theorem 1 (IND-ROR security). *Let P denote the proposed protocol. Under the random-oracle heuristic for H and F_{PUF} , the IND-CPA security of E_k , and the freshness conditions of the ROR model, the adversary's authenticated key exchange advantage in the ROR sense against P is negligible in the security parameter:*

$$\text{Adv}_{P,\mathcal{A}}^{\text{ror-ake}}(\lambda) \leq c_1 \epsilon_{\text{hash}} + c_2 \epsilon_{\text{puf}} + c_3 \epsilon_{\text{enc}} + \text{negl}(\lambda),$$

for some absolute constants c_1, c_2, c_3 .

Proof. The theorem is proven through a sequence of games, starting from the real protocol. Let S_i be the event that the adversary $\mathcal{A}$ correctly guesses the bit b in the Test query of Game i .

- **Game 0.** This is the real attack on protocol P . By definition,

$$\text{Adv}_{P,\mathcal{A}}^{\text{ror-ake}} = |2 \cdot \Pr[S_0] - 1|. \quad (7)$$

- **Game 1.** Replace the real hash function $h(\cdot)$ and the real SSL-PUF interface by the random oracles H and F_{PUF} . By the Difference Lemma,

$$|\Pr[S_1] - \Pr[S_0]| \leq \text{Adv}_{\text{Hash}}^{\text{RO}}(\mathcal{A}) + \text{Adv}_{\text{PUF}}^{\text{RO}}(\mathcal{A}). \quad (8)$$

- **Game 2.** Keep the random oracles from Game 1 and modify how S and UD_i handle the secret shares of the polynomial $f(x)$. In encryptions, replace the real shares with random values. Since these shares are always encrypted under keys derived from a PUF-based secret via HKDF, distinguishing encryptions of real shares from encryptions of random values would break IND-CPA. Hence,

$$|\Pr[S_2] - \Pr[S_1]| \leq \text{Adv}_{\text{Enc}}^{\text{IND-CPA}}(\mathcal{A}). \quad (9)$$

- **Game 3.** Modify only the response to the Test query. Instead of computing

$$\text{SSK} = H(ID_i \parallel ID_S \parallel pf' \parallel n_3 \parallel n_4),$$

return a uniformly random string R_{rand} of the same length. In Game 2, the input pf' to H depends on the random polynomial $f'(\cdot)$ and the PUF response R'_i , which are unknown to $\mathcal{A}$. Therefore, the output of H at that input is independent of the adversary's view, and

$$|\Pr[S_3] - \Pr[S_2]| \leq \epsilon_{\text{ind}}. \quad (10)$$

- **Conclusions.** In Game 3, the Test output is uniform, so $\Pr[S_3] = \frac{1}{2}$. By the triangle inequality,

$$\text{Adv}_{P,\mathcal{A}}^{\text{ror-ake}} = |2 \cdot \Pr[S_0] - 1| \leq 2 \cdot (\text{Adv}_{\text{Hash}}^{\text{RO}} + \text{Adv}_{\text{PUF}}^{\text{RO}} + \text{Adv}_{\text{Enc}}^{\text{IND-CPA}} + \epsilon_{\text{ind}}), \quad (11)$$

which is negligible under the stated assumptions.

□

4.2. Informal Security Analysis

In this section, an informal security analysis is provided to demonstrate the resilience of the proposed DLR-Auth scheme against a variety of well-known attacks. It is argued that the unique combination of chaotic synchronous PUFs, secret sharing, and a carefully designed protocol flow ensures comprehensive security.

4.2.1. Resistance to Replay Attacks

A replay attack occurs when an adversary captures messages from an earlier session and replays them. The scheme prevents replay by using per-session freshness values [19]. In the authentication phase, UD_i sends a fresh nonce n_0 and S replies with n_1 . The session key SSK_{iS} is derived using two additional nonces n_3 and n_4 . Because n_0, n_1, n_3, n_4 are unique to the session, messages from past sessions contain stale nonces and fail verification. For multi-access, each request carries a timestamp T_k that the server accepts only if $(T_k - T_C) < \Delta T$. Requests with outdated timestamps are rejected. Thus, replayed messages are not accepted.

4.2.2. Resistance to Impersonation Attacks

An impersonation attack refers to an act where an adversary attempts to masquerade as a legitimate user or server. To impersonate UD_i , the adversary must successfully complete the authentication phase. This requires the adversary to ultimately generate a valid verifier (e.g., $E_{t+4} = h(SSK_{iS} || \dots)$), which depends on whether the adversary knows the correct SSK . The calculation of SSK relies on the secret template pf' . The template $pf' = f(R'_i)$ is derived from two secret pieces of information that the adversary cannot obtain: (i) the secret polynomial $f(x)$, whose shares are distributed and protected; (ii) the PUF response R'_i , which is generated locally by the physical twin SSL-PUF in response to a fresh challenge and is never transmitted.

To impersonate S , the adversary needs to correctly respond to the initial access request from UD_i . This involves providing correctly encrypted secret shares and polynomial coefficients. Without access to the server database storing user records and the Master SSL-PUF used to derive foundational secrets for encryption, the above operations cannot be completed. If the adversary has physical access to the device and the twin SSL-PUF, they may impersonate S ; otherwise, impersonation is not feasible.

4.2.3. Resistance to Man-in-the-Middle (MITM) Attacks

In a man-in-the-middle (MITM) attack, the adversary is located between UD_i and S . The adversary relays messages between them and may alter the messages.

The protocol prevents this attack by securely binding the session key to the identities of legitimate participants. The session key is calculated as $SSK_{iS} = h(ID_i || ID_S || pf' || \dots)$. Both the user identity ID_i and the server identity ID_S are hashed into the session key.

An MITM attacker cannot simply forward verification messages between the two parties. This is because the hash values will not match if the attacker does so.

4.2.4. Resistance to Stolen SC/Device Attacks

This section analyzes the scenario where an adversary physically captures the device of the user.

If the SC is stolen, the adversary can access a subset of the polynomial's secret shares. However, due to the (t, n) -threshold property of the SSS, possessing fewer than t shares does not reveal any information about the secret polynomial $f(x)$ or the template pf [26]. Therefore, the adversary cannot reconstruct critical secrets using only the SC.

If the device is compromised (without the SC), the adversary may access credentials in the OTP memory, such as ID_i and SV_{iS} . However, the adversary cannot generate the correct

response R'_i to the server's challenge without the *twin* SSL-PUF, which is a physically unclonable component.

Even with both the SC and the device, the adversary cannot pass authentication. They would have the secret shares and long-term key, but without the twin SSL-PUF, they cannot produce the correct R'_i . This leads to an incorrect template pf' , an invalid SSK, and failed authentication. The security of the protocol relies on the unclonable nature of the hardware, which cannot be extracted.

4.2.5. Anonymity and Untraceability

The protocol ensures user privacy through two mechanisms.

In the initial access request, the user's true identity ID_i is never transmitted in plaintext. It is masked in a pseudonym, where the pseudonym is calculated as $PID_i = ID_i \oplus h(n_0 \parallel SV_{iS})$. If an eavesdropping adversary does not know the long-term shared key SV_{iS} , the adversary cannot compute ID_i from the intercepted message.

Furthermore, the adversary cannot link different sessions to the same user. This is because every new session uses fresh random nonces (n_0, n_1, n_3, n_4) . These fresh nonces make the transmitted messages computationally indistinguishable between different sessions.

Additionally, after each successful authentication, the public identifier ID_{SC} is updated to $ID'_S = ID_S \oplus n_4$. This dynamic update prevents the adversary from tracking a user through a static identifier across multiple sessions. The level of untraceability achieved is comparable to that of protocols designed specifically for this purpose [32].

4.2.6. Forward and Backward Secrecy

The baseline construction does not provide strict forward secrecy.

Each session key SSK is derived using per-session nonces (n_3, n_4) and a fresh PUF challenge–response pair (C_{auth}, R'_i) . This derivation method ensures session uniqueness and restricts key reuse. Under the assumptions of nonce and PUF response freshness, and the security of HKDF, leaking a past session key cannot help an adversary predict future session keys.

Strict forward secrecy requires that per-session key material is independent of any long-term secret. If a long-term secret or the recovery material stored by the server is compromised, the baseline design cannot meet this requirement. This aspect is not within the scope of the baseline design.

5. Performance Analysis

This section presents a performance analysis of the proposed DLR-Auth scheme. The evaluation quantifies computational overhead and communication cost, and assesses suitability for resource-constrained IIoT settings. Results are compared with representative authentication protocols [19,22,24] under a common measurement setup.

5.1. Computational Overhead Analysis

This section evaluates the computational performance of DLR-Auth. The analysis has two parts. First, a theoretical comparison breaks down the cryptographic operations and contrasts them with representative schemes. Second, a practical execution-time study applies measured timings from the literature to the protocol workflow to estimate performance.

The following notation is used for major operations and for advanced primitives reported in recent work:

- T_h : time for a cryptographic hash.
- T_{PUF} : time for an SSL-PUF challenge–response.

- T_{SSS_rec} : time to reconstruct a secret from t shares in Shamir's scheme (e.g., via Lagrange interpolation).
- T_{poly} : time to evaluate a degree- t polynomial.
- $T_{enc/dec}$: time for symmetric encryption/decryption.
- $T_{PQSIG_gen/ver}$: time to generate/verify a post-quantum signature.
- $T_{ZKP_gen/ver}$: time to generate/verify a zero-knowledge proof.

5.1.1. Comparison of Computational Complexity

This section compares the computational cost of the primary authentication phase with representative schemes from 2023 to 2024. The cost is reported as counts of cryptographic operations on the user side (UD_i) and the server side (S). Table 4 lists the per-party operation counts and the detailed comparison.

Table 4. Comparison of computational complexity in authentication phase.

Scheme	User-Side (UD_i) Overhead	Server-Side (S) Overhead
Yu et al. (2024) [24]	$1 \cdot T_{FER} + 1 \cdot T_{PUF} + 15 \cdot T_h + \dots$	$1 \cdot T_{VAN} + 16 \cdot T_h + \dots$
Gao et al. (2025) [20]	$2 \cdot T_{PUF} + 1 \cdot T_{RNG} + 1 \cdot T_{enc} + \dots$	$2 \cdot T_{PUF} + 1 \cdot T_{RNG} + 1 \cdot T_{dec} + \dots$
Zhong et al. (2023) [33]	$1 \cdot T_{PUF} + 1 \cdot T_{ZKP_gen} + \dots$	$1 \cdot T_{ZKP_ver} + \dots$
Mathews and Panchami (2024) [34]	$1 \cdot T_{PUF} + 1 \cdot T_{PQSIG_gen} + 4 \cdot T_h + \dots$	$1 \cdot T_{PQSIG_ver} + 4 \cdot T_h + \dots$
Our Scheme (Auth) (this work)	$1 \cdot T_{PUF} + (2t + 7) \cdot T_h + 1 \cdot T_{poly} + (t + 2) \cdot T_{dec}$	$1 \cdot T_{PUF} + (2t + 11) \cdot T_h + 1 \cdot T_{SSS_rec} + (t + 2) \cdot T_{enc}$
Our Scheme (Multi-Access) (this work)	$2 \cdot T_h$	$2 \cdot T_h$

Table 4 shows that recent schemes adopt more complex cryptographic primitives to counter new threats. For example, Mathews and Panchami (2024) [34] use a post-quantum signature (T_{PQSIG}) to resist quantum attacks, and Zhong et al. (2023) [33] employs zero-knowledge proofs (T_{ZKP}) so a device can prove possession of its PUF response without revealing it, improving privacy. These primitives are computationally intensive, especially the generation steps (T_{ZKP_gen} , T_{PQSIG_gen}), which must run on the resource-constrained device.

In contrast, the full authentication phase of the scheme has moderate complexity, relying mainly on symmetric-key primitives (hashing and symmetric encryption) and fast hardware operations (PUF, SSS). The Multi-Access phase incurs only two hash evaluations.

Thus, relative to recent trends, the scheme is efficient for frequent-access scenarios—an aspect often underemphasized by protocols that prioritize broader properties such as post-quantum resistance or zero-knowledge privacy.

5.1.2. Empirical Execution-Time Analysis

To map theoretical complexity to practical runtime, empirical measurements are used. Per-primitive execution times are taken from the experimental setup of Yu et al. [24], which reflects a resource-constrained device. Table 5 lists the reference timings.

Table 5. Execution time of primitive operations.

Operation Symbol	Operation Description	Average Execution Time (ms)
T_h	SHA-256 Hash Operation	0.07
T_{PUF}	Arbiter PUF Response	0.13
T_{SSS_rec} (T_{VAN})	SSS Secret Reconstruction	0.58
T_{poly} (T_{EP})	Polynomial Evaluation (Exponentiation)	0.65
$T_{enc/dec}$	Symmetric Encryption/Decryption	~1.0 (Average)

Note. The Arbiter PUF timing is used as a conservative estimate for the SSL-PUF component; dedicated hardware may be faster. For symmetric primitives, encryption and decryption times are averaged.

Using these timings, the runtime cost of each phase of DLR-Auth is computed. SSS with threshold $t = 3$ is assumed, so reconstruction requires three shares and the polynomial degree is $t - 1 = 2$. Table 6 reports the per-party execution time for the main protocol phases.

Table 6. Calculation of protocol execution time.

Phase & Participant	Calculation Breakdown (Operations × Time per Operation)	Total (ms)
Full Authentication	UD_i	$1 \cdot T_{PUF} + 13 \cdot T_h + 1 \cdot T_{poly} + 5 \cdot T_{dec} =$ $1 \cdot (0.13) + 13 \cdot (0.07) + 1 \cdot (0.65) + 5 \cdot (1.0) = 6.69$
	S	$1 \cdot T_{PUF} + 17 \cdot T_h + 1 \cdot T_{SSS_{rec}} + 5 \cdot T_{enc} =$ $1 \cdot (0.13) + 17 \cdot (0.07) + 1 \cdot (0.58) + 5 \cdot (1.0) = 6.90$
Multi-Access	UD_i	$2 \cdot T_h = 2 \cdot (0.07) = 0.14$
	S	$2 \cdot T_h = 2 \cdot (0.07) = 0.14$

The final estimated execution times are summarized in Table 7.

Table 7. Final estimated execution time.

Protocol Phase	Participant	Average Execution Time (ms)
Full Authentication	UD_i	6.69
	S	6.90
Multi-Access	UD_i	0.14
	S	0.14

Empirical measurements show that a full mutual authentication run completes in 13.59 ms. These values are obtained from the primitive-level time data in Table 6 and should be regarded as optimistic lower limits for comparable resource-constrained devices. This is because they do not account for network latency and other software or operating system overheads that exist in actual deployments. The Multi-Access phase requires 0.28 ms, which is an approximate $49\times$ reduction compared with a full re-authentication. These timings indicate that the scheme reduces the overhead of frequent, repeated access to server resources. Figure 5 compares the cumulative execution time of DLR-Auth with that of representative authentication protocols as the number of consecutive authenticated accesses increases. For the baseline schemes, the cumulative cost grows almost linearly because each access requires a full mutual authentication run. In contrast, DLR-Auth performs one full run to establish the secure context and then uses the lightweight Multi-Access procedure for subsequent accesses, so the cumulative curve of DLR-Auth becomes significantly flatter after the first access.

5.2. Communication Overhead Analysis

This section analyzes the communication overhead of the DLR-Auth protocol. Communication cost efficiency is critical for protocols deployed in bandwidth-constrained IIoT environments. The analysis has two parts. First, the number of interactions and the size of messages exchanged in each phase of the protocol are detailed. Second, a quantitative comparison is provided against the communication costs of related state-of-the-art schemes, including recent works from 2023 to 2024.

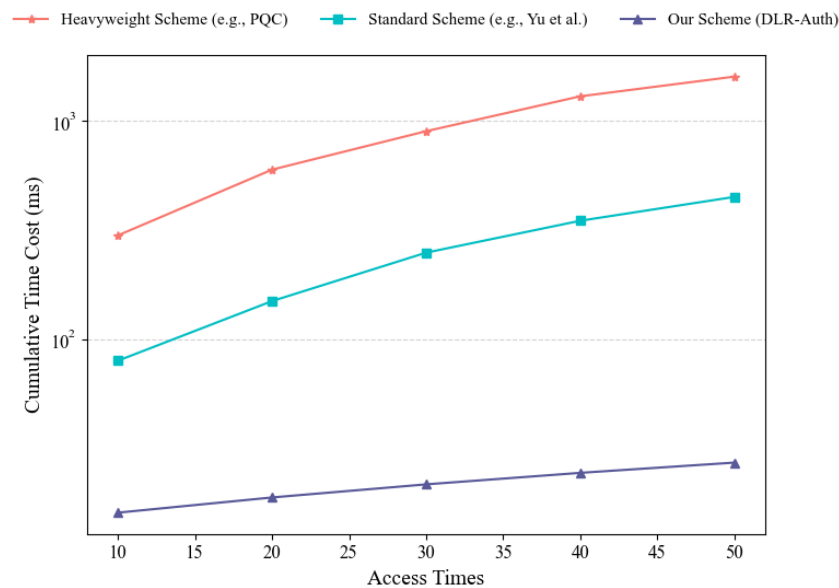


Figure 5. Comparison of cumulative time cost for different authentication schemes [14].

For this analysis, standard assumptions on bit lengths are maintained: identifiers and nonces are 128 bits, hash outputs are 256 bits, and timestamps are 32 bits.

5.2.1. Analysis of Protocol Message Size and Interactions

This section computes the communication overhead of each phase of *DLR-Auth* based on the flow in Chapter 4.

(1) Registration: This phase uses four transmissions. The $UD_i \rightarrow S$ request $\langle ID_S, n_0, PID_i \rangle$ has size $128 + 128 + 256 = 512$ bits. The $S \rightarrow UD_i$ reply $\langle C_{reg}, C_{data} \rangle$ includes a 256-bit PUF challenge and an encrypted payload of size L_{data} determined by the SSS parameters. The final confirmation from UD_i is a 256-bit hash. The one-time cost is dominated by L_{data} .

(2) Full Authentication: This phase has four message flows. The initial request $\langle ID_S, n_0, PID_i \rangle$ is 512 bits. The server response (encrypted shares and polynomial coefficients) is the largest term and depends on the threshold t . The hardware challenge C_{auth} is 256 bits. The final verifier from UD_i , $\langle E_{t+3}, E_{t+4} \rangle$, is $128 + 256 = 384$ bits. Under typical parameters, the total per run is approximately 3000–4000 bits, comparable to other full authentication schemes.

(3) Multi-Access: This phase uses a single message. The request $\langle T_k, E_{t+5}, E_{t+6} \rangle$ contains a timestamp, a ciphertext, and a hash verifier, totaling $32 + 128 + 256 = 416$ bits. This single message constitutes the entire communication for a subsequent access.

5.2.2. Analysis of Communication Expenses

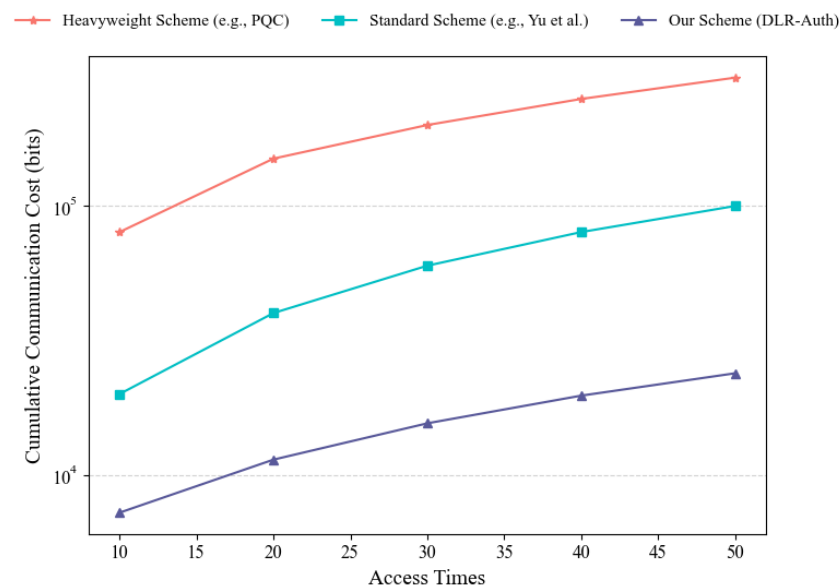
This subsection compares the communication cost of *DLR-Auth* with related protocols, including the privacy-preserving scheme of Zhong et al. (2023) [33] and the post-quantum scheme of Mathews and Panchami (2024) [34]. The number of interactive rounds and the total transmitted bits are reported in the updated Table 8.

The updated results in Table 8 summarize the communication cost of recent schemes. The protocol of Zhong et al. (2023) [33], which uses zero-knowledge proofs (ZKPs), achieves strong privacy but incurs a communication cost of about 3200 bits due to the proof size and other message fields. The post-quantum scheme of Mathews and Panchami (2024) [34] illustrates the large overhead of lattice-based PQC: signatures and public keys are on the order of several kilobytes, so the total communication cost is roughly one order of magnitude higher than that of classical schemes.

Table 8. Comparison of communication costs.

Scheme	Interaction Rounds	Total Communication Cost (Bits)
Yu et al. (2016) [14]	5	3904
Zheng et al. (2022) [19]	3	2368
Banerjee et al. (2021) [22]	2	864
Zhong et al. (2023) [33]	3	~3200 (Estimated)
Mathews and Panchami (2024) [34]	2	>20,000 (Estimated, PQC)
Our scheme (Full Auth) (this work)	4	~3500 (Estimated)
Our scheme (Multi-Access) (this work)	1	416

The comparison in Table 8 shows that the communication cost of the full authentication phase of DLR-Auth (approximately 3500 bits) is comparable to classical schemes with similar security goals. The Multi-Access phase, in contrast, uses a single round and 416 bits, and is explicitly optimized for high-frequency access scenarios, where most recent protocols do not provide an equally lightweight design. Overall, DLR-Auth balances modern security requirements with low communication overhead for practical IIoT deployments. Figure 6 illustrates the communication cost differences among the compared schemes.

**Figure 6.** Comparison of cumulative communication cost for different authentication schemes [14].

6. Conclusions

This paper presented DLR-Auth, a secure, efficient, and decentralized authentication framework for resource-constrained IIoT environments. The work first identified key challenges in existing schemes, including privacy risks associated with biometrics, security bottlenecks in centralized architectures, and the overhead of repeated authentications in frequent-access scenarios. To address these issues, DLR-Auth was designed as a two-party authentication protocol that operates directly between UD_i and S , without a trusted third-party registration center. The scheme combines chaotic synchronous PUFs with SSS to support secure key management and credential revocability. Formal security analysis and performance evaluation indicate that DLR-Auth achieves a practical balance between security, revocability, and operational efficiency in IIoT deployments.

Several limitations remain and suggest directions for future research. First, the security of the scheme depends on the physical integrity of the SSL-PUF hardware, which makes it vulnerable to direct physical tampering [35]. Second, the reliability of the chaotic synchronization mechanism can be affected by extreme environmental conditions, such

as large fluctuations in temperature and supply voltage. Third, the current design of DLR-Auth relies on classical cryptographic primitives and is not resistant to quantum-capable adversaries.

These limitations motivate several extensions. The protocol could be combined with tamper-evident packaging or on-chip intrusion detection to strengthen resistance against physical attacks. Further study of SSL-PUF device characteristics, together with improved reconciliation algorithms, may enhance robustness over a wider range of operating conditions. In addition, adapting the DLR-Auth framework to post-quantum cryptography, while preserving its lightweight properties, is an important direction for future work.

Author Contributions: Y.D.: Writing-original draft, Methodology. Y.L.: Visualization. Y.Y.: Validation. X.G.: Supervision. C.B.: Data curation, Validation. M.L.: Investigation. All authors have read and agreed to the published version of the manuscript.

Funding: This research was supported by the Fundamental Research Funds for the Central Universities (Grant Numbers: 3282025042; 3282023009).

Data Availability Statement: The original contributions presented in the study are included in the article, further inquiries can be directed to the corresponding authors.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

PUF	Physical Unclonable Function
SSL-PUF	Superlattice Synchronization-Based PUF
DLR-Auth	Decentralized Lightweight Revocable Authentication
UD_i	User Device
S	Server
IIoT	Industrial Internet of Things
SSS	Shamir's Secret Sharing
RC	Registration Center
ROR	Real-or-Random
TTPs	Trusted Third Parties
ECC	Elliptic-Curve Cryptography
XOR	Exclusive-OR

References

1. Boyes, H.; Hallaq, B.; Cunningham, J.; Watson, T. The industrial internet of things (IIoT): An analysis framework. *Comput. Ind.* **2018**, *101*, 1–12. [[CrossRef](#)]
2. Ryu, H.; Kim, H. Privacy-Preserving Authentication Protocol for Wireless Body Area Networks in Healthcare Applications. *Healthcare* **2021**, *9*, 1114. [[CrossRef](#)]
3. El-Meniawy, N.; Rizk, M.R.; Ahmed, M.A.; Saleh, M. An Authentication Protocol for the Medical Internet of Things. *Symmetry* **2022**, *14*, 1483. [[CrossRef](#)]
4. Wang, S.; Fan, Z.; Su, Y.; Zheng, B.; Liu, Z.; Dai, Y. A Lightweight, Efficient, and Physically Secure Key Agreement Authentication Protocol for Vehicular Networks. *Electronics* **2024**, *13*, 1418. [[CrossRef](#)]
5. Liu, Y.; Li, C.; Sun, Q.; Jiang, H. Security Authentication Scheme for Vehicle-to-Everything Task Offloading in Edge-Computing Based Autonomous Driving Systems. *Sensors* **2025**, *25*, 6428. [[CrossRef](#)]
6. May, M.C.; Glatter, D.; Arnold, D.; Pfeffer, D.; Lanza, G. IIoT System Canvas—From architecture patterns towards an IIoT development framework. *J. Manuf. Syst.* **2024**, *72*, 437–459. [[CrossRef](#)]
7. Sarkar, A.; Khan, M.Z.; Noorwali, A. Chaos-guided neural key coordination for improving security of critical energy infrastructures. *Complex Intell. Syst.* **2021**, *7*, 2907–2922. [[CrossRef](#)]

8. Panchal, A.C.; Khadse, V.M.; Mahalle, P.N. Security issues in IIoT: A comprehensive survey of attacks on IIoT and its countermeasures. In Proceedings of the 2018 IEEE Global Conference on Wireless Computing and Networking (GCWCN), Lonavala, India, 23–24 November 2018; IEEE: New York, NY, USA, 2018; pp. 124–130.
9. Rührmair, U.; Sehnke, F.; Sölter, J.; Dror, G.; Devadas, S.; Schmidhuber, J. Modeling attacks on physical unclonable functions. In Proceedings of the 17th ACM Conference on Computer and Communications Security, Chicago IL, USA, 4–8 October 2010; pp. 237–249.
10. Hazratifard, M.; Gebali, F.; Mamun, M. Using Machine Learning for Dynamic Authentication in Telehealth Environments. *Sensors* **2022**, *22*, 7655. [\[CrossRef\]](#)
11. Lee, H.J.; Kook, S.; Kim, K.; Ryu, J.; Lee, H.; Lee, Y.; Won, D. Lightweight and Efficient Authentication and Key Distribution Scheme for Cloud-Assisted IoT for Telemedicine. *Sensors* **2025**, *25*, 2894. [\[CrossRef\]](#)
12. Yuan, M.; Xiao, Y. PMAKA-IoV: A Physical Unclonable Function (PUF)-Based Multi-Factor Authentication and Key Agreement Protocol for Internet of Vehicles. *Information* **2025**, *16*, 404. [\[CrossRef\]](#)
13. Mbachu, U.M.; Fatima, R.; Sherif, A.; Dockery, E.; Mahmoud, M.; Alsabaan, M.; Khalil, K. Hardware Acceleration-Based Privacy-Aware Authentication Scheme for Internet of Vehicles Using Physical Unclonable Function. *Sensors* **2025**, *25*, 1629. [\[CrossRef\]](#)
14. Yu, M.D.; Hiller, M.; Delvaux, J.; Sowell, R.; Devadas, S.; Verbauwhede, I. A lockdown technique to prevent machine learning on PUFs for lightweight authentication. *IEEE Trans.-Multi-Scale Comput. Syst.* **2016**, *2*, 146–159. [\[CrossRef\]](#)
15. Plusquellic, J. Shift register, reconvergent-fanout (sirf) puf implementation on an fpga. *Cryptography* **2022**, *6*, 59. [\[CrossRef\]](#)
16. Huang, Z.; Zeng, F.; Chi, Y.; Lin, Y.; Lu, Y.; Liang, H.; Bian, J.; Ouyang, Y.; Ni, T.; Wen, X. BF PUF: A Modeling Attack-Resistant Strong PUF Based on Bent Functions. *IEEE Trans. Very Large Scale Integr. (Vlsi) Syst.* **2025**, *33*, 2299–2311. [\[CrossRef\]](#)
17. Gao, Y.; Ma, H.; Al-Sarawi, S.F.; Abbott, D.; Ranasinghe, D.C. PUF-FSM: A controlled strong PUF. *IEEE Trans.-Comput.-Aided Des. Integr. Circuits Syst.* **2017**, *37*, 1104–1108. [\[CrossRef\]](#)
18. Chatterjee, U.; Govindan, V.; Sadhukhan, R.; Mukhopadhyay, D.; Chakraborty, R.S.; Mahata, D.; Prabhu, M.M. Building PUF based authentication and key exchange protocol for IoT without explicit CRPs in verifier database. *IEEE Trans. Dependable Secur. Comput.* **2018**, *16*, 424–437. [\[CrossRef\]](#)
19. Zheng, Y.; Liu, W.; Gu, C.; Chang, C.H. PUF-based mutual authentication and key exchange protocol for peer-to-peer IoT applications. *IEEE Trans. Dependable Secur. Comput.* **2022**, *20*, 3299–3316. [\[CrossRef\]](#)
20. Gao, X.; Chen, B.; Yao, X.; Yuan, Y.; Suo, Z. SSL-FL: A lightweight authentication framework for secure federated learning. *J. King Saud Univ. Comput. Inf. Sci.* **2025**, *37*, 23. [\[CrossRef\]](#)
21. Yu, K.; Tan, L.; Yang, C.; Choo, K.K.R.; Bashir, A.K.; Rodrigues, J.J.; Sato, T. A blockchain-based shamir’s threshold cryptography scheme for data protection in industrial internet of things settings. *IEEE Internet Things J.* **2021**, *9*, 8154–8167. [\[CrossRef\]](#)
22. Banerjee, S.; Das, A.K.; Chattopadhyay, S.; Jamal, S.S.; Rodrigues, J.J.; Park, Y. Lightweight failover authentication mechanism for IoT-based fog computing environment. *Electronics* **2021**, *10*, 1417. [\[CrossRef\]](#)
23. Minwalla, C.; Plusquellic, J.; Tsiropoulou, E.E. Lightweight Mutually Authenticated Key Exchange with Physical Unclonable Functions. *Cryptography* **2024**, *8*, 46. [\[CrossRef\]](#)
24. Yu, X.; Zhang, K.; Suo, Z.; Wang, J.; Wang, W.; Zou, B. An efficient authentication scheme syncretizing physical unclonable function and revocable biometrics in Industrial Internet of Things. *J. King Saud-Univ.-Comput. Inf. Sci.* **2024**, *36*, 102166. [\[CrossRef\]](#)
25. Mousavi, A.; Baraniuk, R.G. Learning to invert: Signal recovery via deep convolutional networks. In Proceedings of the 2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), New Orleans, LA, USA, 5–9 March 2017; IEEE: New York, NY, USA, 2017; pp. 2272–2276.
26. Faraoun, K.M. A novel fast and provably secure (t, n)-threshold secret sharing construction for digital images. *J. Inf. Secur. Appl.* **2014**, *19*, 331–340. [\[CrossRef\]](#)
27. Kulkarni, K.; Lohit, S.; Turaga, P.; Kerviche, R.; Ashok, A. Reconnet: Non-iterative reconstruction of images from compressively sensed measurements. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, 27–30 June 2016; pp. 449–458.
28. Canh, T.N.; Jeon, B. Multi-scale deep compressive sensing network. In Proceedings of the 2018 IEEE Visual Communications and Image Processing (VCIP), Taichung, Taiwan, 10–12 December 2018; IEEE: New York, NY, USA, 2018; pp. 1–4.
29. Zhang, J.; Ghanem, B. ISTA-Net: Interpretable optimization-inspired deep network for image compressive sensing. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, USA, 18–23 June 2018; pp. 1828–1837.
30. Rührmair, U.; Sölter, J.; Sehnke, F. On the foundations of physical unclonable functions. *IACR Cryptol. ePrint Arch.* **2009**, 277. Available online: <https://eprint.iacr.org/2009/277> (accessed on 13 November 2025).
31. Shi, W.; Jiang, F.; Liu, S.; Zhao, D. Scalable convolutional neural network for image compressed sensing. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Long Beach, CA, USA, 15–20 June 2019; pp. 12290–12299.
32. Che, W.; Martin, M.; Pocklassery, G.; Kajuluri, V.K.; Saqib, F.; Plusquellic, J. A privacy-preserving, mutual PUF-based authentication protocol. *Cryptography* **2016**, *1*, 3. [\[CrossRef\]](#)

33. Zhong, Y.; Hovanes, J.; Guin, U. On-Demand Device Authentication using Zero-Knowledge Proofs for Smart Systems. In Proceedings of the Great Lakes Symposium on VLSI 2023 (GLSVLSI '23), Knoxville, TN, USA, 5–7 June 2023; pp. 569–574. [\[CrossRef\]](#)
34. Mathews, M.M.; Panchami, V. QS-Auth: A Quantum-secure mutual authentication protocol based on PUF and Post-Quantum Signature for Heterogeneous Delay-Tolerant Networks. *J. Inf. Secur. Appl.* **2024**, *83*, 103787. [\[CrossRef\]](#)
35. Delvaux, J.; Peeters, R.; Gu, D.; Verbauwhede, I. A survey on lightweight entity authentication with strong PUFs. *ACM Comput. Surv. (CSUR)* **2015**, *48*, 1–42. [\[CrossRef\]](#)

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.